

Проект (П_1_1), модуль 1, вторая попытка

Выполняя Проект, вы будете работать самостоятельно дома.

Что от вас потребуется?	Для чего?
Самостоятельно поработать с документацией по языку C#, в т.ч. осуществлять информационный поиск	Чтобы закрепить и продемонстрировать, что вы умеете пользоваться поиском и работать с документацией по языку C#
Разработать программы, определённые общей задачей и индивидуальным вариантом	Чтобы продемонстрировать, что вы можете решить достаточно большие задачи, качественно декомпозировать их решения по статическим методам и оформлять код на языке C# согласно требованиям
Сдать заархивированную папку, содержащую решение (Solution), содержащее два проекта с решениями задач в SmartLMS вовремя.	Чтобы показать, что вы умеете создавать решение в IDE Visual Studio, размещать там проекты. Мы уверены, что пользоваться архиватором вы и так умеете 😊

Что надо сдать?

Сдаём заархивированную папку с решением (Solution).

Когда надо сдать?

Дата и время сдачи указаны в SmartLMS.

Куда надо сдать?

Загружаем архив в решение, как ответ на задание в SmartLMS.

Теоретические материалы для самостоятельной работы

Для записи данных в текстовый файл изучите материалы и примеры кода, представленные по ссылкам в этом разделе.

Для организации чтения данных из файла воспользуйтесь методом `ReadAllLines()`:

- <https://learn.microsoft.com/ru-ru/dotnet/api/system.io.file.readalllines?view=net-6.0>

Для разделения строк на подстроки воспользуйтесь методом `Split()`:

- <https://learn.microsoft.com/ru-ru/dotnet/csharp/how-to/parse-strings-using-split>

Для записи данных в файл воспользуйтесь методами `WriteAllLines()`, `WriteAllText()`:

- <https://learn.microsoft.com/ru-ru/dotnet/api/system.io.file.writealllines?view=net-6.0>
- <https://learn.microsoft.com/ru-ru/dotnet/api/system.io.file.writealltext?view=net-6.0>

Описание задачи

Что в целом надо сделать? Сделать программу, которая создаёт одномерные массивы и размещает в них данные, прочитанные из текстового файла **input.txt**. Эта же программа обрабатывает массивы по правилам, определённым в индивидуальном варианте, и сохраняет данные в выходной текстовый файл **output.txt**. Выходных файлов может быть несколько (для дополнительной задачи), тогда к имени **output** через дефис дописывается номер файла (например, output-1.txt, output-2.txt и т.д.). Формат записи данных, т.е. как записывать текст в файлах, определяется индивидуальным вариантом.

Основная задача

Цикл повторения решения. Предполагается, что программа предлагает пользователю повторять решение до тех пор, пока пользователь этого требует. То есть, после вывода результатов работы в файл программа не закрывается, а спрашивает пользователя, хочет ли он закрыть программу и начать всё с начала. Перед перезапуском решения, например, пользователь может изменить данные входного файла, а решение уже должно воспользоваться новыми данными.

Пример 1. Пример цикла повторения решения

Например, можно воспользоваться циклом `do...while()` и информацией о клавишах клавиатуры, которую можно сохранять в структуры с типом `ConsoleKeyInfo`.

```
/*
 * Дисциплина: "Программирование"
 * Группа: 777ПИ
 * Студент: Сидоров Иван
 * Дата: 01.09.2024
 * Задача: Пользователем с клавиатуры вводятся два вещественных
 * числа a и b. Вычислить и вывести на экран их среднее геометрическое
 */
public class Program
{
    public static void Main()
    {
        ConsoleKeyInfo keyToExit; // Переменная для сохранения ключа выхода.
        double a, // Первое число.
               b; // Второе число.

        do // Цикл повторения решения.
        {
            // Блок ввода данных.
            Console.Write("Введите число a: ");
            double.TryParse(Console.ReadLine(), out a);
            Console.Write("Введите число b: ");
            double.TryParse(Console.ReadLine(), out b);

            // Блок обработки данных.
            double geomMean = Math.Sqrt(a * b);

            // Вывод данных и запрос на повтор решения.
            Console.WriteLine($"Среднее геометрическое равно: {geomMean:f3}");
            Console.WriteLine("Для выхода нажмите Escape....");
            keyToExit = Console.ReadKey();
        }
    }
}
```

```
    } while (keyToExit.Key != ConsoleKey.Escape); // Окончание цикла решения.  
  }  
}
```

Предложение решение можно дополнительно усовершенствовать.

Ввод данных. В индивидуальных вариантах предполагается получение некоторых данных от пользователя с клавиатуры. Если пользователь вводит некорректные данные требуется попросить его повторить ввод только некорректного значения, а не всех данных с самого начала.

Файл **input.txt** (имя файла фиксировано, формат utf-8) содержит данные для одного решения задачи, т.е. при перезапусках решения на одних и тех же данных, ваши результаты изменяться не должны. **Данные в файлах всегда присутствуют, но могут содержать некорректные значения.**

Пример 2. Как может выглядеть входной файл?

Если предполагается получить данные для двух массивов, то данные в файле представлены двумя строками: первая содержит данные для первого массива, вторая – для второго.

Выглядеть содержимое **input.txt** может так:

```
12 100 45 89 0 0 -89 6 18  
1 1 1 0 0 3 -3 -1 0 1
```

Или так:

```
100;203;56  
12;10;-13;44;-2;0;
```

Для одного массива строка в файле будет одна.

Формат строк в файле **input.txt** определён индивидуальным вариантом. Для чтения данных из файла разработайте статический метод.

Обработка данных массивов определяется индивидуальным вариантом. Для обработки данных массивов разработайте статические методы. Обратите внимание, что в вариантах указаны имена массивов и переменных, вы можете заменить их на удовлетворяющие требованиям качества программного кода на языке C#.

Вывод данных. Результаты работы программы записываются в текстовый файл **output.txt** (имя файла фиксировано, формат utf-8). Предполагается, что каждый раз при повторении решения данные в файле обновляются, старые результаты затираются. Для записи данных в файл разработайте статический метод.

Пример 3. Как может выглядеть выходной файл?

Например, если требовалось вычислить два значения и разместить в файле, то содержимое **output.txt** может выглядеть так:

```
101 1001
```

Или так:

```
11;34
```

А может и так:

10

19

Если требовалось разместить одно значение, то число в файле будет одно и т.д.

Ограничения по работе с данными файлов

Предполагается, что файлы со входными данными не очень велики и отлично загружаются в оперативную память. **Потоковые чтение и запись файлов использовать запрещено.**

Для работы с файлами и проверки существования / возможностей доступа к файлу используйте класс **File** (<https://learn.microsoft.com/ru-ru/dotnet/api/system.io.file?view=net-6.0>).

Если потребуется работать с путями к файлам, используйте класс **Path** (<https://learn.microsoft.com/ru-ru/dotnet/api/system.io.path?view=net-8.0>).

Требования к методам:

- Все методы статические и размещены в статических классах того же проекта, где и точка входа в программу.
- В методах отсутствуют действия, не имеющие отношения к алгоритму преобразования входных данных в выходные.
- Методы должны максимально соответствовать принципам единственной ответственности, т.е. выполнять один алгоритм, а не несколько. *Например, если при помощи метода вы удаляете из строки какие данные, то этот же метод не должен заниматься выводом на экран или подсчётом символов.*

Требуется обработать исключения:

- Исключения, связанные с наличием / отсутствием входных файлов на диске. Если файл отсутствует на диске, ваша программа не завершает работу, а выводит пользователю сообщение: «Входной Файл на диске отсутствует», если отсутствует файл **input.txt**. После чего предлагает пользователю перезапустить решение задачи с начала.
- Исключения, связанные с открытием / закрытием и чтением / записью данных из файлов на диске. Если возникают проблемы с записью или чтением, ваша программа не завершает работу, а выводит пользователю одно из сообщений:
 - «Проблемы с открытием файла»;
 - «Проблемы с сохранением файла»;
 - «Проблемы с чтением данных из файла»;
 - «Проблемы с записью данных в файл».

После чего предлагает пользователю перезапустить решение задачи с начала.

- Исключения, связанные с приведением и переполнением типов. Если происходят ошибки приведения или переполнения типов, ваша программа не должна завершаться аварийно или просто прекращать работу, требуется вывести оповещение

пользователю «Данные неверны» и попросить повторить ввод только некорректных данных, как это указано выше в задании.

Дополнительная задача

Для перехода к решению дополнительной задачи обязательная задача должна быть полностью выполнена в указанных в ней ограничениях. Для дополнительной задачи ограничения не расширяются и не отменяются. Оформите дополнительную задачу отдельным проектом в том же решении, которое будете сдавать.

Ввод данных. В рамках дополнительной задачи предполагается, что файл **input.txt** (имя файла фиксировано, формат utf-8) содержит несколько строк данных, которые последовательно необходимо обработать программой за один сеанс работы. Для чтения данных разработайте статические методы. **Данные в файлах всегда присутствуют, но могут содержать некорректные значения.**

Пример 4. Как может выглядеть входной файл со многими данными?

Если предполагается получить данные для двух массивов, то данные в файле представлены парами строк, первая в паре содержит данные для первого массива, вторая – для второго.

Выглядеть это может так:

```
12 100 45 89 0 0 -89 6 18
1 1 1 0 0 3 -3 -1 0 1
```

```
13 67 90 234
10 11 12 12 0 -89
```

Или так:

```
100;203;56
12;10;-13;44;-2;0

1;1; 1; 0; 0; 3; -3; -1; 0; 1

13;67;90;234
```

Количество строк всегда корректно.

Вывод данных. В рамках дополнительной задачи предполагается, что результаты сеанса работы программы записываются в текстовый файл **output-HOME.txt** (формат utf-8). При этом после каждого запуска HOME увеличивается на единицу. Программа записывает последний номер файла **output-** в специальный текстовый файл **config.txt** (формат utf-8) и ориентируется при создании следующего файла на него. Для записи данных в файл и работы с **config.txt** разработайте статические методы. Файл **config.txt** разместите в папке решения, вне папок всех проектов, содержащихся с этим решением.

Пример 5. Как может выглядеть один из выходных файлов?

Например, если требовалось вычислить два значения и разместить в файле, то содержимое **output-....txt** может выглядеть так:

```
101 1001
55 96
```

Или так:

11;34

1000;5

А может и так:

10

19

100

85

Индивидуальные варианты

Вариант 16

Программа получает из **input.txt** данные для двух массивов. Данные в строках **input.txt** вещественные (десятичный разделитель - запятая) и разделены ровно одной точкой с запятой без дополнительных пробелов. В файле могут присутствовать некорректные данные, их следует пропустить, не выводя оповещений пользователю, если данных при этом не останется, пользователю выводится сообщение «Корректных данных в файле нет» и предлагается повторить решение сначала.

Массивы A и B должны состоять из одинакового количества N элементов. Если это не так, программа выводит сообщение «Корректных данных в файле нет» и предлагает повторить решение сначала. По данным из A и B требуется вычислить значения D_1, D_2

$$D_1 = \max |a_i - b_i|, i = 0, \dots, N - 1$$

$$D_2 = \min |a_i - b_i|, i = 0, \dots, N - 1$$

D_1 и D_2 программа сохраняет в **output.txt**, размещая их в одной строке и разделяя одним пробелом. Точность вывода данных в файл – три знака после запятой.

Все файлы разместить в папке с запускаемым файлом проекта.

Вариант 17

Программа получает из **input.txt** данные для двух массивов. Данные в строках **input.txt** вещественные (десятичный разделитель - запятая) и разделены ровно одним пробелом. В файле могут присутствовать некорректные данные, их следует пропустить, не выводя оповещений пользователю, если данных при этом не останется, пользователю выводится сообщение «Корректных данных в файле нет» и предлагается повторить решение сначала.

Массив A состоит из N элементов, массив B из M элементов. По данным из A и B требуется вычислить значения S_A и S_B . Для обработки массивов используйте статические методы.

$$S_A = \sum_{k=0}^{N-1} A_k$$

$$S_B = \sum_{k=0}^{M-1} B_k$$

В **output.txt** программа сохраняет S_A и S_B и значение отношения $\min(S_A; S_B) / \max(S_A; S_B)$, предварительно вычисляя их. В файле значения размещаются в одной строке и разделяются точкой с запятой без дополнительных пробелов. Точность вывода данных в файл – два знака после запятой.

Файлы разместить в отдельной папке **WorkingFiles**, расположенной на уровне основной папки проекта.

Вариант 18

Программа получает из **input.txt** данные для двух массивов. Данные в строках **input.txt** целочисленные положительные и разделены одной точкой с запятой без дополнительных пробелов. Напоминаем, что в файле могут присутствовать некорректные данные, их следует пропустить, не выводя оповещений пользователю, если данных при этом не останется, пользователю выводится сообщение «Корректных данных в файле нет» и предлагается повторить решение сначала.

Массивы X и Y должны состоять только из значений строго больше 0 меньше 10000, кратных пяти, в каждом массиве по N элементов. Если данные в файлах таковы, что один из массивов больше другого или пуст, то вычисления не производятся, и в результирующий файл записывается значение -1, пользователю выводится сообщение «Корректных данных в файле нет» и предлагается повторить решение сначала. По данным массивов одинаковой длины требуется вычислить значение S . Для обработки массивов используйте статические методы.

$$S = \sum_{k=0}^{N-1} \frac{1}{X_k} + \frac{1}{Y_k}$$

В **output.txt** программа сохраняет минимальное и максимальное значение из X и Y , а также значение S . Точность вывода вещественных данных в файл – четыре знака после запятой. Данные разделяются строго одним пробелом и размещаются в одной строке.

Все файлы разместить непосредственно в папке с решением (Solution) выше уровня всех проектов.